Week 5 — The Hackathon: Vibe Coding a Production Simulator

What This Week Is

No guided exercises. No step-by-step walkthrough. This week you ship software.

Over the past four weeks you learned:

- How LLMs work under the hood (Week 1)
- The OpenAI API, context windows, and how to build an API server (Week 2)
- Function calling and tool-use patterns (Week 3)
- Multi-round tool loops, dispatch tables, and error handling (Week 4)

Now you put it all together. You will work in **teams of 2–4 students** to vibe-code a complete application from a product specification — a **3D Printer Production Simulator**. The twist: your primary coding tool is **Claude Code**, the Anthropic CLI agent, and you must use it deliberately and well.

This session is the kickoff. You will get the project running today; you continue on your own time until the deadline.

Part 1: Setting Up Claude Code

Claude Code is a terminal-based AI coding agent made by Anthropic. It reads your codebase, proposes changes, runs commands, and iterates — all from your terminal. Think of it as a pair programmer that lives in your shell.

Step 1 — Install Claude Code

```
npm install -g @anthropic-ai/claude-code
```

If you don't have `npm`, install Node.js first:

```
# Ubuntu/Debian
sudo apt install nodejs npm

# macOS (with Homebrew)
brew install node
```

Verify the installation:

```
claude --version
```

Step 2 — Configure the API Endpoint

You will **not** use Anthropic's API directly. We provide you with an endpoint and API key. Set them as environment variables before launching Claude Code.

Add these to your shell profile (`~/.bashrc`, `~/.zshrc`, or equivalent):

```
export ANTHROPIC_API_KEY="the-key-we-give-you"
export ANTHROPIC_BASE_URL="the-endpoint-we-give-you"
```

Then reload:

```
source ~/.bashrc   # or source ~/.zshrc
```

The instructor will give you the actual values at the start of the session. **Do not commit these to git.** If you want to store them per-project, put them in a `.env` file and add `.env` to your `.gitignore` — but note that Claude Code reads environment variables, not `.env` files directly.

Step 3 — Verify It Works

Navigate to any directory and run:

```
claude
```

You should see Claude Code's interactive prompt. Type a simple question like "What is FastAPI?" and confirm you get a response. If you get an authentication error, double-check your environment variables.

Step 4 — Learn the Basics

Key commands inside Claude Code's interactive mode:

Command	What it does
Just type naturally	Ask questions, request code changes, describe what you want
`/help`	Show available commands
`/status`	Show current context and file state
`/clear`	Clear conversation context
`/compact`	Compress the conversation to free up context window
`Ctrl+C`	Cancel current generation
`Ctrl+D` or `/exit`	Exit Claude Code

Claude Code can:

- Read and edit files in your project
- Run shell commands (with your approval)
- Create new files
- Search your codebase
- Run tests

It asks for confirmation before making changes. Read what it proposes before accepting.

Part 2: The PRD-First Workflow

"Vibe coding" does not mean "start typing and hope." It means **using AI deliberately**, with a plan. The plan is your **PRD — Product Requirements Document**.

Why a PRD?

When you vibe-code without a plan, every prompt is disconnected. The AI has no memory of what you decided earlier. It invents architecture on the fly. You end up with a Frankenstein codebase.

A PRD solves this. It is a document that captures:

- What you are building
- What the data model looks like
- What the architecture is
- What libraries you are using and why
- What the key interfaces are

When you feed the PRD to Claude Code (via a `CLAUDE.md` file or by referencing it), every coding session starts from the same shared understanding.

How to Build the PRD with Claude Code

This is your first task as a team. Before writing any application code, you will use Claude Code to draft and refine the PRD.

1. Create your project repository:

```
mkdir printer-factory-sim
cd printer-factory-sim
git init
```

2. Launch Claude Code and start the conversation:

```
claude
```

3. Give it the project brief. Paste or describe the project specification (see Part 3 below). Then ask Claude Code to help you draft a PRD. A good opening prompt:

> "I need to build a 3D Printer Production Simulator. Here is the specification: [paste the spec]. Help me create a Product Requirements Document (PRD) that covers: the data model, the architecture, the tech stack decisions, the API endpoints, and a development plan with milestones. Ask me clarifying questions before drafting."

4. Iterate. Claude Code will ask questions. Answer them as a team. Discuss the trade-offs. Push back when you disagree. This is the most important part — the PRD is your team's consensus, not the AI's first draft.

5. Save the PRD:

Once you are happy with the PRD, save it as `docs/PRD.md` in your repository.

6. Create a `CLAUDE.md` file in your project root. This file is special — Claude Code reads it automatically every time it starts in your project directory. It should contain:

```
# Project: 3D Printer Production Simulator

## What This Is
[One-paragraph summary from your PRD]

## Tech Stack
- Python 3.11+
- FastAPI + Pydantic for the REST API
- Streamlit for the dashboard UI
- SQLite for persistence
- [SimPy for discrete event simulation — if you chose it]
- matplotlib for charts

## Architecture
[Key architecture decisions from your PRD]

## Data Model
[Summary of your entities and relationships]

## Coding Conventions
- Use type hints everywhere
- Pydantic models for all API request/response schemas
- Keep API routes in separate files from business logic
- Write docstrings for public functions
- All configuration via environment variables or config files

## Current State
[Update this as you progress — what is done, what is next]
```

This file is your **living contract** with Claude Code. Every time you start a new session, Claude Code reads it and knows where you are.

The PRD Is a Deliverable

Your PRD is not a throwaway document. It is part of your final submission. A good PRD shows that you thought before you coded.

Part 3: The Project — 3D Printer Production Simulator

Objective

Build a software system that simulates, day by day, the full production cycle of a factory that manufactures 3D printers. The focus is on **inventory management, purchasing, and production planning**. The user plays the role of production planner: they decide what to manufacture and what to buy.

Functional Requirements (Minimum)

R0 — Initial Configuration

Define the production plan: - Bill of Materials (BOM) for each printer model (what raw materials are needed, in what quantities) - Assembly time per model - Supplier catalog: products for sale, prices by quantity (e.g., per pallet of 1000 units, per box of 20), lead time for delivery - Warehouse capacity (simplified: 1 unit of any material = 1 storage unit)

R1 — Demand Generation

At the start of each simulated day, manufacturing orders are generated randomly. Parameters (mean and variance) must be configurable.

R2 — Control Dashboard

Display: - Pending orders - Bill of Materials breakdown for each order - Current inventory levels

R3 — User Decisions

The user must be able to: - Release orders to production - Issue purchase orders (choose product, supplier, quantity, and date)

R4 — Event Simulation

- Raw material consumption during manufacturing, limited by daily capacity
- Purchase arrivals according to supplier lead time

R5 — Calendar Advance

A button "Advance Day" runs the 24h simulation cycle.

R6 — Event Log

Record all events for historical tracking and charts.

R7 — JSON Import/Export

Export and import inventory state and event history as JSON.

R8 — REST API

Every piece of functionality and information shown in the UI must be accessible via a REST API documented with Swagger / OpenAPI.

Non-Functional Requirements

- Clean, commented code versioned with Git
- Simple web interface; no complex client-side installation
- Cross-platform (Windows, macOS, Linux)

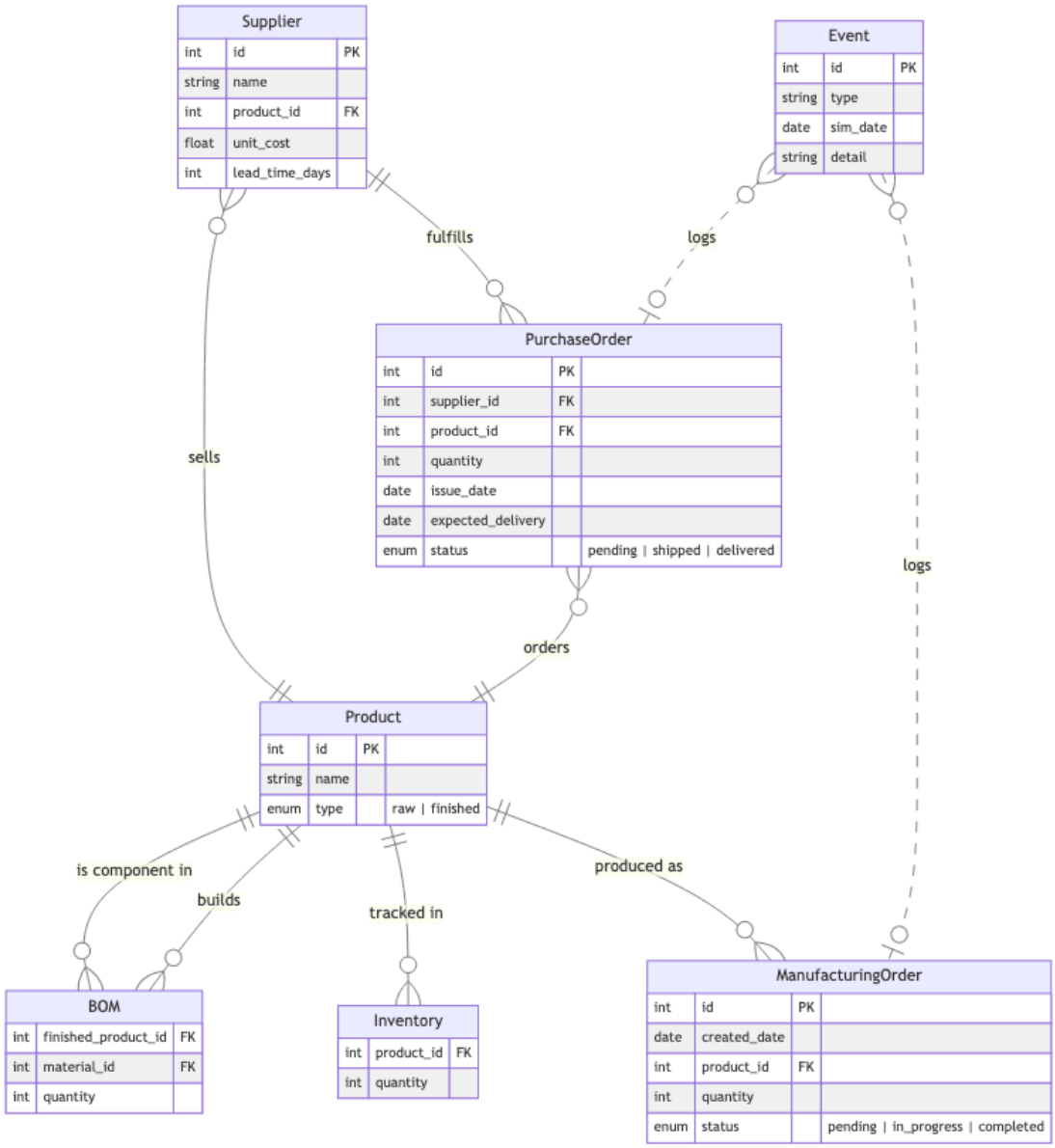
Proposed Tech Stack

Layer	Tool	Reason
Language	Python 3.11+	Widely used, simple syntax
Simulation	SimPy (recommended)	Discrete event simulation engine, easy to use
Persistence	SQLite and/or JSON files	Lightweight and portable
Back-end / API	FastAPI + Pydantic	REST API with automatic OpenAPI docs
UI	Streamlit	Fast dashboard construction
Charts	matplotlib	Direct integration with Streamlit
Version control	Git + GitHub	Standard workflow

SimPy is recommended but not mandatory. If your team prefers a different simulation approach (e.g., a simple turn-based loop), that is fine — justify it in your PRD.

Initial Data Model (Approximate)

```
Product(id, name, type)                    # "raw" or "finished"
BOM(finished_product_id, material_id, qty) # bill of materials
Supplier(id, name, product_id, unit_cost, lead_time_days)
Inventory(product_id, quantity)
ManufacturingOrder(id, created_date, product_id, quantity, status)
PurchaseOrder(id, supplier_id, product_id, quantity, issue_date,
expected_delivery, status)
Event(id, type, sim_date, detail)          # all events logged
```



Example Production Plan

```
{
  "capacity_per_day": 10,
  "models": {
    "P3D-Classic": {
      "bom": {
        "kit_piezas": 1,
        "pcb": 1,
        "pcb_ref": "CTRL-V2",
        "extrusor": 1,
```

```
        "cables_conexion": 2,
        "transformador_24v": 1,
        "enchufe_schuko": 1
      }
    },
    "P3D-Pro": {
      "bom": {
        "kit_piezas": 1,
        "pcb": 1,
        "pcb_ref": "CTRL-V3",
        "extrusor": 1,
        "sensor_autonivel": 1,
        "cables_conexion": 3,
        "transformador_24v": 1,
        "enchufe_schuko": 1
      }
    }
  }
}
```

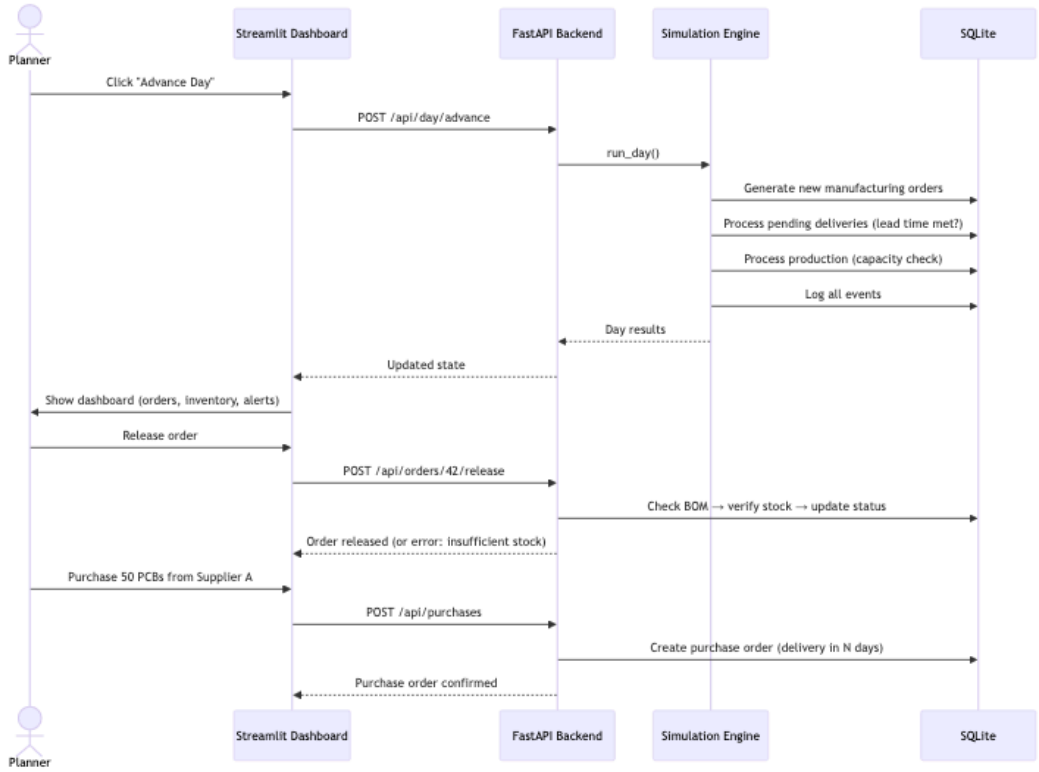

Example Scenario

- **Day 1:** Starting stock = 30 parts kits. Capacity = 10 printers/day. The system generates two orders: 8 and 6 units. The planner releases only the order for 8. The system checks stock (8 kits consumed). 22 remain. No purchase needed yet.
- **Day 2:** Two new orders arrive: 5 and 7 units. Stock is running low. The planner decides to buy 20 kits from Supplier A (cost: 90 EUR/kit, lead time: 3 days).
- **Day 5:** The purchased kits arrive. Production can resume at full capacity.

The cycle continues. The interesting part is the tension between demand, inventory, capacity, and lead times.

Daily Simulation Flow

1. **Advance Day** → simulation engine runs 24h
2. New manufacturing orders are generated
3. The dashboard updates; the user makes decisions
4. The simulation processes purchases and production
5. End of day: all events are recorded



Minimum UI Layout

- **Header:** Current simulated day + "Advance Day" button
- **Orders panel:** Table of pending orders with automatic BOM calculation
- **Inventory panel:** Current stock levels and shortages
- **Purchasing panel:** Supplier dropdown, quantity field, "Issue Order" button
- **Production panel:** Daily capacity, orders in queue and in progress
- **Charts:** Stock levels and completed orders over time

Part 4: Development Workflow

Phase 1 — Today's Session (The Kickoff)

By the end of today's 2-hour session, your team should have:

1. **Repository created** on GitHub with all team members as collaborators
2. **Claude Code working** for every team member
3. **PRD drafted** (`docs/PRD.md`) — data model, architecture, tech stack decisions, milestones
4. **`CLAUDE.md` created** in the project root
5. **GitHub Issues created** for your milestones/checkpoints
6. **Skeleton code running** — at minimum, a FastAPI server that starts and a Streamlit app that loads

This is ambitious for two hours. Prioritise the PRD and the skeleton.

Phase 2 — After Today (On Your Own Time)

Continue building toward the full specification. Use Claude Code as your primary tool. Work through the milestones you defined in your PRD.

Recommended development order:

1. Data model in Python (Pydantic models)
2. Core simulation logic (day advancement, demand generation)
3. Basic Streamlit dashboard showing inventory and orders
4. User actions: release orders, issue purchase orders
5. BOM calculation and stock verification
6. Supplier lead times and purchase arrivals
7. REST API with FastAPI (documented with Swagger)
8. Charts and historical data
9. JSON import/export
10. Polish, error handling, edge cases

Git Discipline

Every time you reach a stable milestone:

```
git add .
git commit -m "feat: implement inventory tracking — closes #3"
```

Reference the GitHub Issue number in the commit message. This creates a traceable link between your plan and your code.

Using Claude Code Effectively

Start each session by grounding Claude Code:

```
> Read CLAUDE.md and docs/PRD.md. Then look at the current codebase.
> Tell me what has been implemented and what the next milestone is.
```

Be specific in your requests:

Bad: "Build the inventory system."

Good: "Implement the Inventory model as a Pydantic class and create a FastAPI endpoint `GET /api/inventory` that returns the current stock levels from the SQLite database. Follow the data model in docs/PRD.md."

Use Claude Code for architecture decisions:

> "I need to decide how to handle the day simulation. Option A is SimPy with a discrete event engine. Option B is a simple function that iterates through pending orders and processes them sequentially. What are the trade-offs? Which is better for a Streamlit-based UI where the user clicks 'Advance Day'?"

Use Claude Code for debugging:

> "The purchase order is not arriving on the expected day. Here is my simulation code. Find the bug."

Use `/compact` when conversations get long. Claude Code has a context window. If you've been working for a while and responses get worse, run `/compact` to summarise the conversation and free up space.

Part 5: Deliverables

1. GitHub Repository

Your repository must contain:

- All source code with a working `README.md` (installation and usage instructions)
- `docs/PRD.md` — your Product Requirements Document
- `CLAUDE.md` — your Claude Code project file
- `.gitignore` (must exclude `.env`, `__pycache__/`, `.venv/`, `*.db`)
- Clean commit history with issue references

2. Report (PDF, 3–5 pages)

Generated from Markdown:

```
pandoc report.md -o report.pdf --pdf-engine=xelatex
```

If you include Mermaid diagrams in your report (recommended for architecture and data model), install the mermaid filter first:

```
npm install -g mermaid-filter
```

Then generate the PDF with the filter:

```
pandoc report.md -o report.pdf --pdf-engine=xelatex -F mermaid-filter
```

Mermaid diagrams render automatically on GitHub and in VS Code / Cursor's markdown preview. The filter is only needed for PDF generation.

The report must include:

a) Design decisions - Data model and why you structured it this way - Architecture choices (why SimPy or why not, why Streamlit, how API and UI interact) - Trade-offs you discussed as a team

b) The PRD process - How you used Claude Code to build the PRD - What you changed from Claude Code's initial suggestions and why - Specific examples of prompts that worked well and prompts that didn't

c) Screenshots of the working application - The dashboard - At least one complete day cycle (advance day, make decisions, see results) - The Swagger/OpenAPI documentation page

d) Analysis of a test scenario - Run the example scenario (or a similar one) for at least 5 simulated days - Show what decisions you made and what happened - Include relevant charts or data

e) Vibe coding reflection - How did you divide work in the team? - What did Claude Code do well? Where did it struggle? - What would you do differently next time? - Did the PRD-first approach help? How?

3. Short Presentation (max 10 slides)

Prepare a short presentation covering: - What you built (live demo if possible) - Key design decisions - Lessons learned from vibe coding with Claude Code

Write It Yourself

We want to read **your** words, not an LLM's.

Using Claude Code to build the software is the whole point. Using an LLM to write your report is not. We want to know what *your team* decided, what broke, what surprised you, and how you recovered.

If your report could have been written by someone who never ran the code, it is not good enough.

Part 6: Milestones and Checkpoints

Identify milestones and create them as **GitHub Issues**. Reference them in your commits.

Here is a suggested (not mandatory) milestone structure:

#	Milestone	Description
1	Project setup	Repo, CLAUDE.md, PRD, dependencies, skeleton
2	Data model	Pydantic models, SQLite schema, seed data
3	Simulation core	Day advance, demand generation, basic production
4	Dashboard v1	Streamlit showing inventory, orders, basic controls
5	User actions	Release orders, issue purchases, BOM checks
6	Supplier logic	Lead times, purchase arrivals, stock updates
7	REST API	FastAPI endpoints with Swagger docs
8	Charts & history	Event log, matplotlib charts in Streamlit
9	Import/export	JSON serialization of state and events
10	Polish	Error handling, edge cases, documentation

Make a commit at each milestone with stable, working code. The commit message must reference the issue number.

Part 7: Technical Hints

These are not instructions. They are clues for when you get stuck.

On SimPy

SimPy models processes as Python generators. A simple day loop might look like:

```
import simpy

def day_cycle(env, factory):
    while True:
        yield env.timeout(1)  # 1 day passes
        factory.generate_demand()
        factory.process_production()
        factory.receive_deliveries()
```

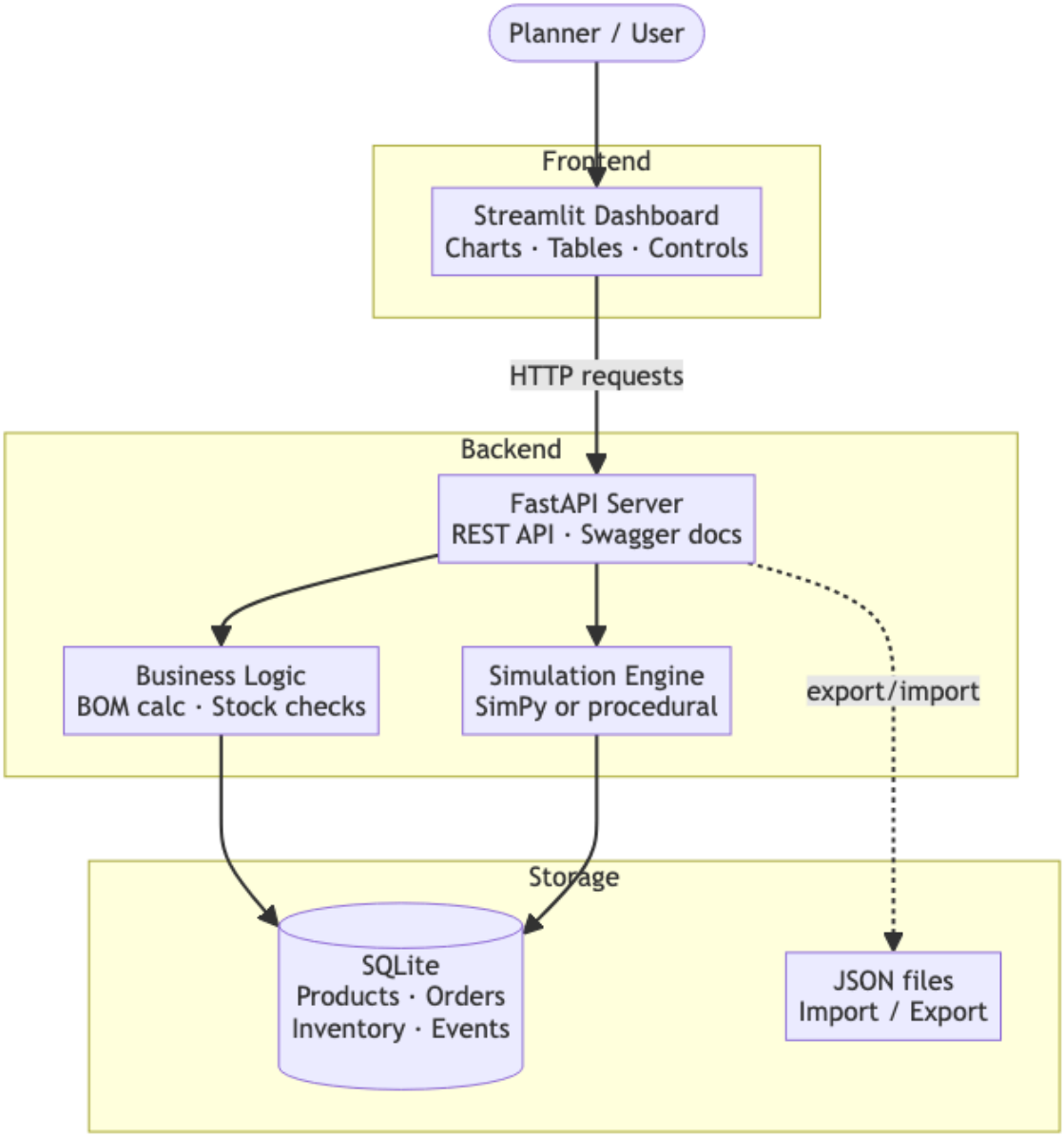
SimPy is powerful but has a learning curve. If your team decides against it, a simple procedural approach works too — a function that processes one day's events in sequence.

On Streamlit + FastAPI

Streamlit and FastAPI can run as separate processes. The Streamlit app calls the FastAPI backend via HTTP. This gives you a clean separation:

- FastAPI owns the data and simulation logic
- Streamlit is a pure display layer

Alternatively, Streamlit can talk directly to a shared Python module. Discuss the trade-offs in your PRD.



On Pydantic Models

Use Pydantic for your data model. It gives you validation, serialization, and automatic JSON Schema — which is exactly what FastAPI needs for Swagger docs.

```
from pydantic import BaseModel
from typing import Literal

class Product(BaseModel):
    id: int
    name: str
    type: Literal["raw", "finished"]
```

```python
class BOMEntry(BaseModel):
    finished_product_id: int
    material_id: int
    quantity: int

class Supplier(BaseModel):
    id: int
    name: str
    product_id: int
    unit_cost: float
    lead_time_days: int
```

On the REST API

FastAPI generates Swagger docs automatically at `/docs`. Make sure every endpoint that powers a UI view is also exposed in the API. The spec says: "Every piece of functionality shown in the UI must be accessible via a REST API."

On Seed Data

Create a JSON file with initial data (products, BOMs, suppliers, starting inventory). Load it on startup. This makes testing reproducible and lets you reset the simulation.

API Key Safety

Same rules as every week. **This is not optional.**

1. Never put API keys in `.py` files
2. Never commit `.env` to git
3. Always have `.env` in `.gitignore` before your first commit
4. Include a `.env.example` with placeholder values

Your `.gitignore` must contain at minimum:

```
.env
__pycache__/
.venv/
*.db
```

Summary

What	How
Teams	2–4 students
Tool	Claude Code CLI, configured with provided endpoint
First step	Draft the PRD with Claude Code
Architecture doc	`CLAUDE.md` + `docs/PRD.md`
Main project	3D Printer Production Simulator
Tracking	GitHub Issues + commit references
Deliverables	GitHub repo + PDF report (3–5 pages) + presentation (10 slides)

This week is about learning to **drive** an AI coding agent on a real project — not just follow along with exercises. The PRD is your steering wheel. Claude Code is the engine. You still need to know where you're going.